B.Tech. CSE / AI & ML - IV Sem

Software Engineering (CS403 / AL403)

Software Engineering:

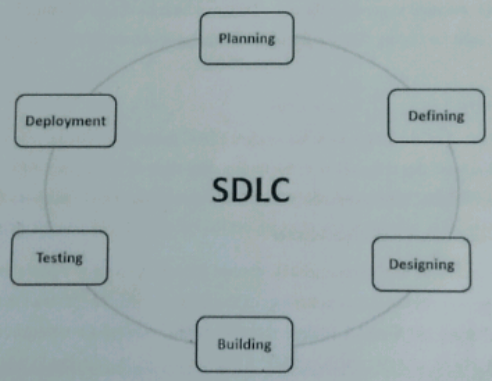
- It is a systematic, disciplined, cost-effective techniques for software development
- Engineering approach to develop a software.
- Software engineering is a discipline within computer science that focuses on the design, development, testing, and maintenance of software applications. It involves applying engineering principles and computer programming expertise to build robust, reliable, and user-friendly software systems.

Software Development Life Cycle:

Software Development Life Cycle (SDLC) is a process used by the software industry to design, develop and test high quality softwares. The SDLC aims to produce a high-quality software that meets or exceeds customer expectations, reaches completion within times and cost estimates.

- SDLC is the acronym of Software Development Life Cycle.
- It is also called as **Software Development Process**.
- SDLC is a framework defining tasks performed at each step in the software development process.

The following figure is a graphical representation of the various stages of a typical SDLC:



A typical Software Development Life Cycle consists of the following stages –

Stage 1: Planning and Requirement Analysis

Requirement analysis is the most important and fundamental stage in SDLC. It is performed by the senior members of the team with inputs from the customer, the sales department, market surveys and domain experts in the industry. This information is then used to plan the basic project approach and to conduct product feasibility study in the economical,

operational and technical areas.

Planning for the quality assurance requirements and identification of the risks associated with the project is also done in the planning stage.

Stage 2: Defining Requirements

Once the requirement analysis is done the next step is to clearly define and document the product requirements and get them approved from the customer or the market analysts. This is done through an SRS (Software Requirement Specification) document which consists of all the product requirements to be designed and developed during the project life cycle.

Stage 3: Designing the Product Architecture

SRS is the reference for product architects to come out with the best architecture for the product to be developed. Based on the requirements specified in SRS, usually more than one design approach for the product architecture is proposed and documented in a DDS - Design Document Specification.

This DDS is reviewed by all the important stakeholders and based on various parameters as risk assessment, product robustness, design modularity, budget and time constraints, the best design approach is selected for the product.

Stage 4: Building or Developing the Product

In this stage of SDLC the actual development starts and the product is built. The programming code is generated as per DDS during this stage. If the design is performed in a detailed and organized manner, code generation can be accomplished without much hassle.

Developers must follow the coding guidelines defined by their organization and programming tools like compilers, interpreters, debuggers, etc. are used to generate the code. Different high level programming languages such as C, C++, Pascal, Java and PHP are used for coding. The programming language is chosen with respect to the type of software being developed.

Stage 5: Testing the Product

This stage is usually a subset of all the stages as in the modern SDLC models, the testing activities are mostly involved in all the stages of SDLC. However, this stage refers to the testing only stage of the product where product defects are reported, tracked, fixed and retested, until the product reaches the quality standards defined in the SRS.

Stage 6: Deployment in the Market and Maintenance

Once the product is tested and ready to be deployed it is released formally in the appropriate market. Sometimes product deployment happens in stages as per the business strategy of that organization. The product may first be released in a limited segment and tested in the real business environment (UAT- User acceptance testing).

Then based on the feedback, the product may be released as it is or with suggested enhancements in the targeting market segment. After the product is released in the market, its maintenance is done for the existing customer base.

SDLC Models:

There are various software development life cycle models defined and designed which are followed during the software development process. These models are also referred as "**Software Development Process Models**". Each process model

follows a Series of steps unique to its type to ensure success in the process of software development.

Following are the most important and popular SDLC models followed in the industry –

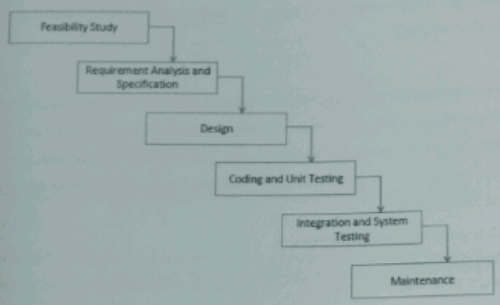
Classical Waterfall Model, Iterative Waterfall Model, V-Shaped Model, Prototyping Model, Incremental Model, Evolutionary Model, Spiral Model, RAD Model, Agile Model, etc.

Classical Waterfall Model:

The classical Waterfall Model was the first Process Model. It is also known as a linear-sequential life cycle model. It is very simple to understand and use.

Classical waterfall model is the earliest, best known and most commonly used methodology. It is a sequential lifecycle that is simple to understand and use each phase has to be completed finished before another start which means no overlapping is allowed.

The waterfall model is intuitively the most obvious way to develop software. It divides the software life cycle into the six phases, as shown in Figure-



1. Feasibility Study

The main aim of the feasibility study is to determine whether it would be technically and financially feasible to develop the product. The feasibility study activity involves analysis of the problem and collection of all relevant information relating to the product.

2. Requirement Analysis and Specification

The aim of the requirements analysis and specification phase is to understand the exact requirements of the customer and to document them properly. This phase consists of two distinct activities, namely requirements gathering and analysis, and requirements specification as follows:

Requirements Gathering and Analysis: The goal of the requirements gathering activity is to collect all relevant information from the customer regarding the product to be developed. Once the requirements have been gathered, the analysis activity is taken up. The goal of the requirement analysis is to weed out the incompleteness and inconsistencies in

these requirements.

Requirement Specification: The customer requirements identified during the requirements gathering and analysis activi are organized into a software requirement specification (SRS) document. The important components of this document are functional requirements, non-functional requirements, and the goals of implementation.

3. Design

The goal of the design phase is to transform the requirements specified in the SRS document into a structure that is suitab for implementation in some programming language. In technical terms, during the design phase, the software architecture derived from the SRS document. There are two design approaches being used at present: traditional design approach and object-oriented design approach.

4. Coding and Unit Testing

The purpose of coding and unit testing phase of software development is to translate the software design into the source code. The coding phase is also sometimes called the **implementation phase** since the design is implemented into a workable solution in this phase. Each component of design is implemented as a program module. The end-product of this phase is a set of program modules that been individually tested. After coding is complete, each module is unit-tested to determine the correct working of all individual modules.

5. Integration and System Testing

During this phase, the different modules are integrated in a planned manner. The plan specifies the order in which modules are combined to realize the full system. Integration of various modules are normally carried out incrementally over a number of steps. After each integration step, the partially integrated system is tested. Finally, after all modules have been successfully integrated and tested, system is carried out. The goal of system testing is to ensure that the developed system conforms to its requirements laid out in the SRS document.

Alpha testing: is the type of testing which is performed by the developer's.

Beta testing: is the type of testing which is performed by a real users and friendly set of customers.

6. Maintenance

Maintenance involves monitoring and improving system performance, enhancing system services, and upgrading to newer versions. Maintenance of a typical software product requires much more effort than the effort necessary to develop the product itself. The past studies indicate that the relative effort of development of a typical software product to its maintenance effort is roughly in 40:60 ratio.

Advantages of Waterfall Model

- Classical waterfall model is easy to understand and simple to use.
- In the waterfall model, only one phase is executed at a time or phases cannot overlap.
- In this model, each phase is clearly defined & each Process, actions and results are well documented.
- Waterfall model works best for a small project, where requirements are clearly defined.

Disadvantages of Waterfall Model

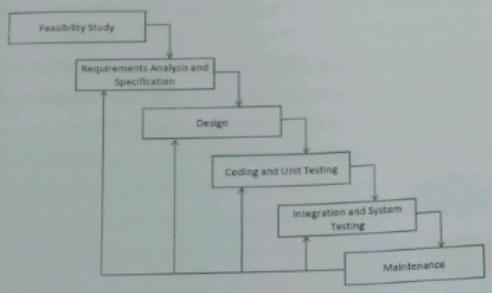
- **Idealistic Model:** The classical waterfall model is an idealistic one since it assumes that no development error is ever committed by the engineers during any of the life cycle phases.

- **Phases are sequential:** Classical waterfall model assumes, that all the phases are sequential. However, that is rarely the case.
- Waterfall model assumes that all requirements are defined correctly at the beginning of the project.

Iterative Waterfall Model:

In practice, it is not possible to strictly follow the classical waterfall model for software development work. In this context, we can view the iterative waterfall model as making necessary changes to the classical waterfall model so that it becomes applicable to practical software development projects.

In Iterative waterfall model, the feedback paths are provided from every phase to its preceding phase as shown in Figure-



The feedback paths allow for correction of the errors committed during a phase, as and when these are detected in a later phase.

For example, if during a testing a design error is identified, then the feedback path allows the design to be reworked and the changes to be reflected in the design documents. However, observe that there is no feedback path to the feasibility stage This means that the feasibility study errors cannot be corrected.

Advantages of Iterative Waterfall Model

- **Feedback Path:** iterative waterfall allows the mechanism of error connection because there is a feedback path fron one phase to its preceding phase which it lacks in the Waterfall Model.
- **Simple:** iterative waterfall model is simple to understand and use. It is the most widely used software development model evolved so far.
- **Parallel development:** can be done.

Disadvantage of Iterative Waterfall Model

- **More resource:** may be required to implement the iterative waterfall model.
- **Difficult to include change requests:** In the iterative waterfall model, all the requirements must be clearly defined before starting of the development phase but sometimes customer requirement changes which is difficult to incorporate change requests that are made after development phase starts.

- **Not support Intermediate delivery:** Project has to be fully completed before it delivered to the customer.
- **Risk handling:** Project is prone to many types of risk but there is no risk handling mechanism.
- Not suitable for a **small project.**

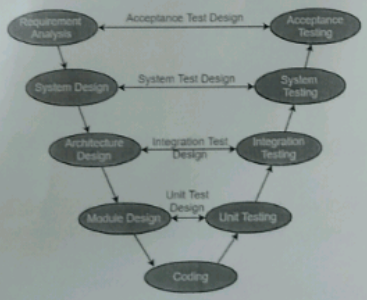
V-Shaped Model:

The V-model is an SDLC model where execution of processes happens in a sequential manner in a V-shape. It is also known as **Verification and Validation model.**

The V-Model is an extension of the waterfall model and is based on the association of a testing phase for each corresponding development stage. This means that for every single phase in the development cycle, there is a directly associated testing phase. This is a highly-disciplined model and the next phase starts only after completion of the previous phase.

Under the V-Model, the corresponding testing phase of the development phase is planned in parallel. So, there are Verification phases on one side of the V and Validation phases on the other side. The Coding Phase joins the two sides of the V-Model.

The following illustration depicts the different phases in a V-Model of the SDLC.



Verification Phases: There are several Verification phases in the V-Model, each of these are explained in detail below.

Requirement Analysis

This is the first phase in the development cycle where the product requirements are understood from the customers perspective. This phase involves detailed communication with the customer to understand his expectations and exact requirement. This is a very important activity and needs to be managed well, as most of the customers are not sure abou what exactly they need. The acceptance test design planning is done at this stage as business requirements can be used input for acceptance testing.

System Design

Once you have the clear and detailed product requirements, it is time to design the complete system. The system design have the understanding and detailing the complete hardware and communication setup for the product under developm The system test plan is developed based on the system design. Doing this at an earlier stage leaves more time for the ac test execution later.

Architectural Design

Architectural specifications are understood and designed in this phase. Usually more than one technical approach is proposed and based on the technical and financial feasibility the final decision is taken. The system design is broken down further into modules taking up different functionality. This is also referred to as High Level Design (HLD).

The data transfer and communication between the internal modules and with the outside world (other systems) is clearly understood and defined in this stage. With this information, integration tests can be designed and documented during this stage.

Module Design

In this phase, the detailed internal design for all the system modules is specified, referred to as Low Level Design (LLD). It is important that the design is compatible with the other modules in the system architecture and the other external systems. The unit tests are an essential part of any development process and helps eliminate the maximum faults and errors at a very early stage. These unit tests can be designed at this stage based on the internal module designs.

Coding Phase

The actual coding of the system modules designed in the design phase is taken up in the Coding phase. The best suitable programming language is decided based on the system and architectural requirements.

The coding is performed based on the coding guidelines and standards. The code goes through numerous code reviews and is optimized for best performance before the final build is checked into the repository.

Validation Phases: The different Validation Phases in a V-Model are explained in detail below.

Unit Testing

Unit tests designed in the module design phase are executed on the code during this validation phase. Unit testing is the testing at code level and helps eliminate bugs at an early stage, though all defects cannot be uncovered by unit testing.

Integration Testing

Integration testing is associated with the architectural design phase. Integration tests are performed to test the coexistence and communication of the internal modules within the system.

System Testing

System testing is directly associated with the system design phase. System tests check the entire system functionality and the communication of the system under development with external systems. Most of the software and hardware compatibility issues can be uncovered during this system test execution.

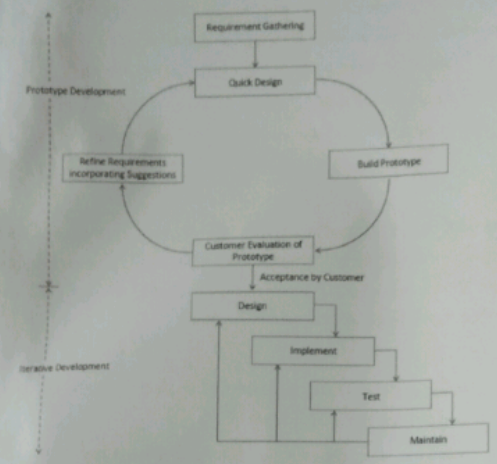
Acceptance Testing

Acceptance testing is associated with the business requirement analysis phase and involves testing the product in user environment. Acceptance tests uncover the compatibility issues with the other systems available in the user environment. It also discovers the non-functional issues such as load and performance defects in the actual user environment.

Prototype Model:

A prototype model is a **toy/demo** implementation of the actual product or system. A prototype model usually exhibits limited functional capabilities, low reliability, and inefficient performance as compared to the actual software.

The prototyping model of software development is shown in Figure.



As shown in Figure, the first phase is prototype development to control various risks. This is followed by an iterative development cycle.

In the prototype model, prototyping starts with initial requirements gathering phase. Quick design is carried out and a prototype is built. The developed prototype is submitted to the customer for his assessment.

Based on the customer feedback, the requirements are refined and the prototype is suitably modified. This cycle of obtaining customer feedback and modifying the prototype continues until the customer approves the prototype. Once the customer approves the prototype, the actual system is developed using the iterative waterfall approach.

Advantages of Prototype Model

- **Demo working model:** Customer get demo working model of actual product which help them to give a better understanding and attain a high level of satisfaction.
- **New requirement:** Based on the customer feedback, the requirements are redefined and the prototype is suitably modified till final approval.
- **Missing functionality:** can be easily established.
- **Easy error detection:** It saves time and cost in developing the prototype and enhance the quality of the final product.
- **Flexibility:** in the development phase.

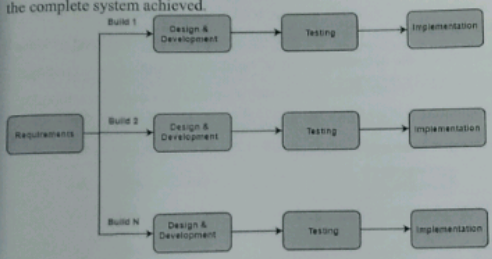
Disadvantages of Prototype Model

- **Time-consuming:** As the prototype is being modified time to time according to customer requirement which usually increases the time of completion of the product.

- **Complexity:** Change in the requirement usually expand the scope of the product beyond its original plan and thus increase the complexity.
- **Poor Documentation:** Continous changing of requirement can lead to poor documentation.
- **Unpredictability of no of iteration:** It is difficult to determine the no of iteration required before the prototype is finally accepted by the customer.
- **Confusion:** Customer can confuse between the actual product and prototype.

Incremental Model:

Incremental Model is a process of software development where requirements divided into multiple standalone modules of the software development cycle. In this model, each module goes through the requirements, design, implementation and testing phases. Every subsequent release of the module adds function to the previous release. The process continues until the complete system achieved.



The various phases of incremental model are as follows:

1. Requirement analysis: In the first phase of the incremental model, the product analysis expertise identifies the requirements. And the system functional requirements are understood by the requirement analysis team. To develop the software under the incremental model, this phase performs a crucial role.

2. Design & Development: In this phase of the Incremental model of SDLC, the design of the system functionality and the development method are finished with success. When software develops new practicality, the incremental model uses style and development phase.

3. Testing: In the incremental model, the testing phase checks the performance of each existing function as well as additional functionality. In the testing phase, the various methods are used to test the behavior of each task.

4. Implementation: Implementation phase enables the coding phase of the development system. It involves the final coding that design in the designing and development phase and tests the functionality in the testing phase. After completion of this phase, the number of the product working is enhanced and upgraded up to the final system product.

Advantage of Incremental Model

- Errors are easy to be recognized.
- Easier to test and debug
- More flexible.
- Simple to manage risk because it handled during its iteration.
- The Client gets important functionality early.

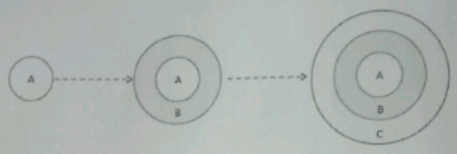
Disadvantage of Incremental Model

- Need for good planning
- Total Cost is high.
- Well defined module interfaces are needed.

Evolutionary Model:

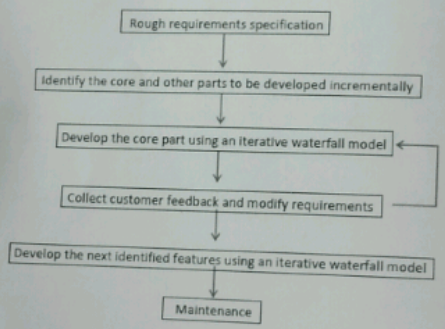
Evolutionary model is a combination of Iterative and Incremental Model of software development life cycle.

Evolutionary model is also referred to as the successive versions model and sometimes as the incremental model. In Evolutionary model, the software requirement is first broken down into several modules (or functional units) that can be incrementally constructed and delivered.



The development first develops the core modules of the system. The core modules are those that do not need services from the other modules. The initial product skeleton is refined into increasing levels of capability by adding new functionalities in successive versions. Each evolutionary model may be developed using an iterative waterfall model of development.

The **evolutionary model** is shown in Figure below. Each successive version/model of the product is a fully functioning software capable of performing more work than the previous versions/model.



The evolutionary model is normally useful for very large products, where it is easier to find modules for incremental implementation.

Often, **evolutionary model** is used when the customer prefers to receive the product in increments so that he can start using the different features as and when they are developed rather than waiting all the time for the full product to be developed and delivered.

Advantages of Evolutionary Model

- **Large project:** Evolutionary model is normally useful for very large products.
- User gets a **chance to experiment with partially developed software** much before the complete version of the system is released.

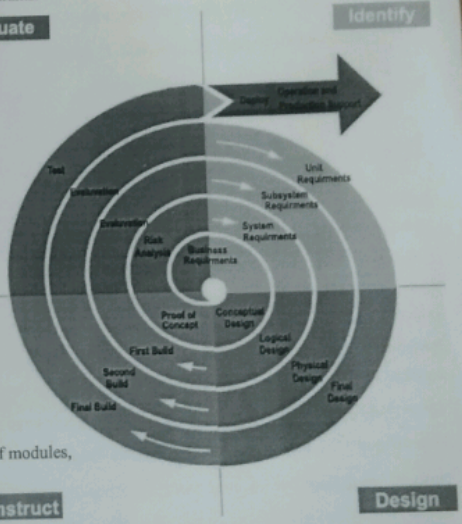
- The core modules get tested thoroughly, thereby **reducing the chances of errors** in the core modules of the final products.
- Evolutionary model **avoids the need to commit large resources** in one go for development of the system.

Disadvantages of Evolutionary Model

- **Difficult to divide the problem into several versions** that would be acceptable to the customer and which can be incrementally implemented and delivered.

Spiral Model:

- It is best suited for complex embedded product development & situations where the requirements are changing from customer's side.
- It supports Risk Handling by evaluating the risks in each stage.
- It is a combination of Waterfall model & Iterative model.
- The exact number of loops of the spiral is unknown and can vary from project to project.
- Each loop of the spiral is called a Phase of the software development process.
- Each phase of Spiral Model is divided into four quadrants.
    - **Identification**
    - **Designing**
    - **Construction**
    - **Evaluation**



IDENTIFICATION

- This phase starts with gathering the requirements.
- This phase also includes understanding the system requirements by continuous communication between the customer and the system analyst.
- The objectives are identified, elaborated and analyzed at the start of every phase.

DESIGNING

- This involves architectural design, logical design of modules, physical product design and the final design in the subsequent spirals.
- At the end of this quadrant, Prototype is built.

CONSTRUCTION

- This phase refers to production of the actual software product.

- The identified features are developed and verified through testing.
- At the end of the third quadrant, a new version of the software is available.
- These builds are sent to the customer for feedback.

EVALUATION

- In the fourth quadrant, the Customers evaluate the so far developed version of the software.
- The software development process enters the next iteration and subsequently follows the linear approach to implement the feedback suggested by the customer.
- The process of iterations along the spiral continues throughout the life of the software.

ADVANTAGES:

- Supports Risk Handling
- Best suited for large projects
- Flexibility in Requirements
- Customer Satisfaction

DISADVANTAGES:

- More complex than other SDLC models
- Not suitable for small projects as it is expensive
- Too much dependable on Risk Analysis
- Difficulty in time management

RAD (Rapid Application Development) Model:

Rapid application development is a software development methodology that uses minimal planning in favor of rapid prototyping. A prototype is a working model that is functionally equivalent to a component of the product.

In the RAD model, the functional modules are developed in parallel as prototypes and are integrated to make the complete product for faster product delivery. Since there is no detailed preplanning, it makes it easier to incorporate the changes within the development process.

RAD projects follow iterative and incremental model and have small teams comprising of developers, domain experts, customer representatives and other IT resources working progressively on their component or prototype.

The most important aspect for this model to be successful is to make sure that the prototypes developed are reusable.

Following are the various phases of the RAD Model –

Business Modeling: The business model for the product under development is designed in terms of flow of information and the distribution of information between various business channels. A complete business analysis is performed to find the vital information for business, how it can be obtained, how and when is the information processed and what are the factors driving successful flow of information.

Data Modeling: The information gathered in the Business Modelling phase is reviewed and analyzed to form sets of data objects vital for the business. The attributes of all data sets is identified and defined. The relation between these data objec

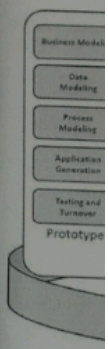
are establishe

Process Mod
information fl
changes or en
or modifying

Application
data models in

Testing and
during every it
with complete
any major issu

The following



The advantag

- Chang
- Progre
- Iterati
- Produ
- Reduc
- Increa
- Encou
- Integr

The disadvan

- Deper

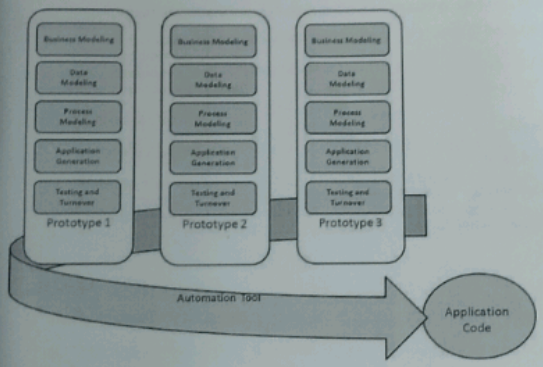
are established and defined in detail in relevance to the business model.

Process Modeling: The data object sets defined in the Data Modelling phase are converted to establish the business information flow needed to achieve specific business objectives as per the business model. The process model for any changes or enhancements to the data object sets is defined in this phase. Process descriptions for adding, deleting, retrievinj or modifying a data object are given.

Application Generation: The actual system is built and coding is done by using automation tools to convert process and data models into actual prototypes.

Testing and Turnover: The overall testing time is reduced in the RAD model as the prototypes are independently tested during every iteration. However, the data flow and the interfaces between all the components need to be thoroughly tested with complete test coverage. Since most of the programming components have already been tested, it reduces the risk of any major issues.

The following illustration describes the RAD Model in detail.



The advantages of the RAD Model are as follows –

- Changing requirements can be accommodated.
- Progress can be measured.
- Iteration time can be short with use of powerful RAD tools.
- Productivity with fewer people in a short time.
- Reduced development time.
- Increases reusability of components.
- Encourages customer feedback.
- Integration from very beginning solves a lot of integration issues.

The disadvantages of the RAD Model are as follows –

- Dependency on technically strong team members for identifying business requirements.

- Only system that can be modularized can be built using RAD.
- Requires highly skilled developers/designers.
- Management complexity is more.
- Suitable for systems that are component based and scalable.
- Requires user involvement throughout the life cycle.
- Suitable for project requiring shorter development times.

Data Flow Diagram:

A **data flow diagram (DFD)** is a graphical representation of the "flow" of data through an information system, modeling its proces aspects.

History:

- In the 1970s, Larry Constantine, the original developer of structured design, proposed data flow diagrams as a practical technique based on Martin and Estrin's "Data Flow Graph" model of computation.
- It became more popular in business circles, as it was applied to business analysis, than in academic circles.

DFD is not a "Flow Chart"

- Flow chart shows "Flow of Control".
- Data flow diagram shows "Flow of Data".
- The flow chart describes boxes that describe "Computation, Decisions, Interactions, & Loops.
- Data flow diagram should not include control elements.

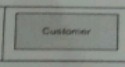
Methods of Symbols used in DFDs

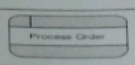
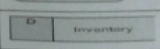
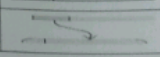
1. Yourdon & Coad
2. Gane & Sarson
3. Yourdon & DeMarco

Difference in Methods:

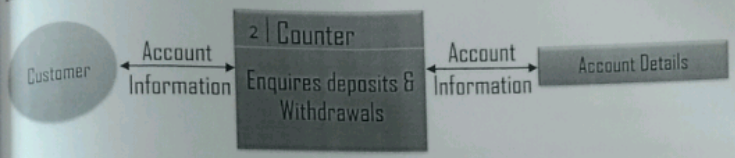
Notation	Yourdon and Coad	Gane and Sarson
External Entity		
Process		
Data Store		
Data Flow		

Components of DFDs

1.	**External Entity**	

2.	**Process**	
3.	**Data Store**	
4.	**Data Flow**	

Example Of a Banking Process:



Rules of Data Flow:

Data Can Flow From	Data Cannot Flow From
• External entity to process • Process to external entity • Process to data store • Data store to process • Process to process	• External entity to external entity • External entity to data store • Data store to External entity • Data store to data store

Types Of Data Flow Diagrams:

1. **Logical Data Flow Diagram**
   - It focuses on the business & how the business operates.
   - It describes the business events that take place & the data required & produced by each event.
   - It does not delve into the technical aspects of a process or a system.
   - Non-technical employees can understand these diagrams easily.
2. **Physical Data Flow Diagram**
   - It shows that the how system will be implemented.
   - These diagrams specify the software, hardware, files & people involved in an information flow.
   - A detailed physical data flow diagram can facilitate the development of the code needed to implement a data system.

DFDs Levels & Layers:

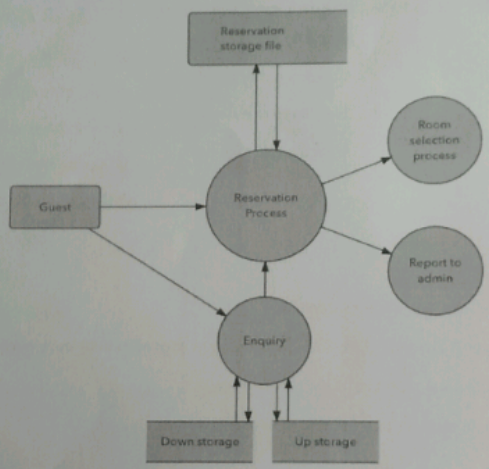
1. **Level 0 – Context Diagram**
   - It is the most basic & simplest Data Flow Diagram.

- It shows a single process node and its connections to external entities.
- This level basically represents the input & output of the entire system.

Example of a Hotel Reservation System



2. **Level 1 – Overview Diagram**
   - Presents more detailed view of the system than Level 0.
   - Highlight the main functions carried out by the system.
   - It is basically the breakdown of level 0 into sub-processes.



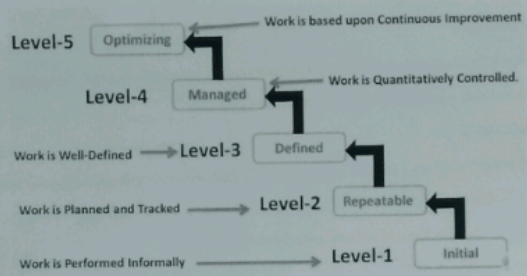
3. **Level 2 – Detailed Diagram**
   - Simply break processes down into more detailed sub-processes.
   - DFDs could go beyond level 3, but they rarely do.
   - Level 3 data flow diagrams are detailed enough that it doesn't usually make sense to break them down furt

B.Tech. CSE / AI & ML - IV Sem

Software Engineering (CS403 / AL403)

Capability Maturity Model (CMM)

- CMM was developed and is promoted by the **Software Engineering Institute (SEI)**, a research and development center.
- It is **not a software process model**. Capability maturity model is used as a benchmark **to measure the maturity** of an organization's software process.
- The model describes a **five-level** evolutionary path of increasingly organized and systematically more mature processes as shown in figure below.



The various levels of SEI-CMM Model are as follows:

1. Level-1 Initial: **Work is performed informally.**
   A software development organization at this level is characterized by **ad hoc activities** (organization is not planned in advance)
2. Level-2 Repeatable: **Work is Planned and Tracked.**
   This level of software development organization has a basic and consistent project management processes to **track cost, schedule, and functionality**. The process is place to **repeat** the earlier successes on projects with similar applications.
3. Level-3 Defined: **Work is Well-Defined.**
   At this level, the software process for both management and engineering activities are **Defined and documented**.
4. Level-4 Managed: **Work is quantitatively controlled.**
   - **Software Quality Management:** Management can effectively control the software development effort using precise measurements. At this level, organization set a quantitative quality goal for both software process and software maintenance.
   - **Quantitative Process Management:** At this maturity level, the performance of processes is controlled using statistical and other quantitative techniques, and is quantitatively predictable.
5. Level-5 optimizing: **Work is based upon continuous improvement.**
   The key characteristics of this level are focusing on **continuously improving process** performance. Key features are:
   - Process change management
   - Technology change management
   - Defect prevention

Examples of few CMM-5 Certified Companies in India:

- Cognizant Technology Solutions
- Infosys Technologies Limited
- Tata Consultancy Services
- Wipro Technologies
- HCL Technologies

Agile Model

The Agile Model in software engineering is a modern and flexible approach to software development that emphasizes **iterative progress, collaboration, and customer feedback.** Unlike traditional models like Waterfall, Agile allows chan and refinements throughout the development process.

Key Characteristics of the Agile Model

1. Iterative and Incremental
   - Development is broken into small, manageable units called iterations or sprints (usually 1–4 weeks).
   - Each sprint delivers a potentially shippable product increment.
2. Customer Collaboration
   - Clients and stakeholders are involved continuously through demos, feedback sessions, and planning meetings.
3. Responding to Change
   - Agile welcomes changes to requirements, even late in development.
4. Cross-Functional Teams
   - Teams consist of developers, testers, designers, and sometimes product owners, all working together.
5. Working Software as a Measure of Progress
   - Functioning code is the main deliverable at the end of each iteration.
6. Continuous Testing and Integration
   - Code is tested regularly to ensure quality and stability.
7. Face-to-Face Communication
   - Communication is frequent and informal, often with daily stand-up meetings (Scrum meetings).

Agile Development Cycle

1. Concept / Requirements Gathering
2. Planning (Sprint Planning)
3. Design
4. Development
5. Testing
6. Demo / Review
7. Release

3. Feedback / Retrospective
2. Repeat the cycle for the next sprint

Popular Agile Frameworks

- **Scrum** – Roles like Product Owner, Scrum Master, and Development Team; uses sprints, backlogs, and stand-up meetings.
- **Kanban** – Visual workflow management; focuses on continuous delivery.

Advantages of Agile Model

- High flexibility and adaptability
- Early and continuous delivery of value
- Greater customer satisfaction
- Better risk management
- Improved team collaboration and morale

Disadvantages of Agile Model

- Less predictable due to changing requirements
- Needs active user involvement
- Can be harder to scale in large teams without proper coordination

SCRUM Model

The Scrum Model is one of the most popular Agile frameworks. It provides a structured yet flexible way to develop complex products by promoting teamwork, accountability, and continuous improvement.

Key Concepts in Scrum

1. Sprints
   - Time-boxed development cycles (usually 1–4 weeks).
   - At the end of each sprint, a potentially shippable product increment is delivered.
2. Roles in Scrum
   - Product Owner
     - Defines and prioritizes features (via the Product Backlog)
     - Represents the voice of the customer
   - Scrum Master
     - Facilitates the process, removes obstacles
     - Ensures adherence to Scrum practices
   - Development Team
     - Cross-functional team that builds the product

Product and Process Metrics

In software engineering, **Product and Process Metrics** are used to measure, analyze, and improve both the quality of the software product and the efficiency of the development process.

Product Metrics

Definition: Product metrics describe the characteristics of the software product itself — such as its quality, complexity, performance, and maintainability.

Common Product Metrics

Metric	Description
Size Metrics	LOC (Lines of Code), Function Points (FP)
Complexity Metrics	Cyclomatic complexity (measures decision points in code)
Quality Metrics	Number of defects, defect density (defects per KLOC)
Performance Metrics	Response time, throughput, resource usage
Reliability Metrics	Mean Time Between Failures (MTBF), system uptime
Maintainability	Time taken to make changes or fix bugs
Reusability	Degree to which code can be reused in other systems

Process Metrics

Definition: Process metrics measure how effective and efficient the software development process is.

Common Process Metrics

Metric	Description
Effort Metrics	Person-hours or person-days spent on a task
Schedule Metrics	Planned vs. actual time for tasks (adherence to deadlines)
Defect Metrics	Number of defects found in various stages (design, coding, testing)
Productivity Metrics	LOC per person-hour, Function Points per developer
Defect Removal Efficiency (DRE)	% of defects removed before release vs. total found
Test Coverage	% of code covered by automated/manual tests

Example:

If you're building an e-commerce app:

Product Metrics: Track page load time, crash rates, number of bugs in checkout.

Process Metrics: Measure how long sprints are taking vs. planned time, how many bugs are found during testing vs. after release.